

Spring Entity Advanced

Anatomy of an Entity (The Annotations)

Here is the breakdown of the essential annotations you will use 90% of the time.

Class Level

- `@Entity` : Marks the class as a JPA entity.
- `@Table(name = "users")` : (Optional) Specifies the table name. If omitted, the class name is used.

Field Level (Primary Key)

- `@Id` : Marks the primary key.
- `@GeneratedValue(strategy = ...)` : How the ID is generated.
 - `GenerationType.IDENTITY` : Auto-increment (MySQL, Postgres).
 - `GenerationType.SEQUENCE` : Uses a database sequence (Oracle, Postgres).
 - `GenerationType.UUID` : Generates a UUID (Hibernate 6+).

Field Level (Columns)

- `@Column` : Customizes column details.
 - `name` : DB column name.
 - `nullable` : `true` / `false` .
 - `unique` : `true` / `false` .
 - `length` : String length (default 255).
- `@Transient` : The field exists in the Java class but **will not** be saved to the database.
- `@Enumerated(EnumType.STRING)` : Saves an Enum as its name ("ACTIVE") instead of its ordinal number (0).

- `@Lob`: Large Object (for long text or blobs).

Imagine an application where we store users, their status (using an Enum), and a large biography (using `@Lob`).

The Enum (Status)

First, we define a simple Enum for the user's account status.

```
public enum AccountStatus {  
    ACTIVE,  
    SUSPENDED,  
    PENDING_VERIFICATION  
}
```

The Entity Example

```
import jakarta.persistence.*;  
import java.util.UUID;  
import lombok.Getter;  
import lombok.Setter;  
import lombok.NoArgsConstructor;  
  
@Entity  
@Table(name = "user_accounts") // Customizes table name to avoid 'user' (SQL reserved word)  
@Getter @Setter @NoArgsConstructor  
public class UserAccount {  
  
    // Primary Key using UUID (Hibernate 6+ style)  
    @Id  
    @GeneratedValue(strategy = GenerationType.UUID)
```

```

private UUID id;

// Custom column mapping with constraints
@Column(name = "full_name", nullable = false, length = 100)
private String fullName;

// Unique constraint for emails
@Column(unique = true, nullable = false)
private String email;

// Saves "ACTIVE" instead of 0 in the database
@Enumerated(EnumType.STRING)
private AccountStatus status;

// Large Object for long text (e.g., a detailed bio)
@Lob
@Column(columnDefinition = "TEXT")
private String bio;

// Exists in Java only (not saved to DB)
@Transient
private String decryptedPassword;

// Example of a default field (will use field name 'age' and default settings)
private int age;
}

```

Breakdown of how this maps to SQL

If you were to look at the database after Hibernate generates the schema, the SQL would look something like this:

Java Field	Database Column	Constraints/Type
id	id	UUID PRIMARY KEY
fullName	full_name	VARCHAR(100) NOT NULL

email	email	VARCHAR(255) UNIQUE NOT NULL
status	status	VARCHAR(255) (stores string value)
bio	bio	TEXT (clob/lob)
decryptedPassword	None	Field ignored by database

2. Handling Relationships

This is often the most complex part of JPA.

A. One-to-One (@OneToOne)

- **Example:** User ↔ UserProfile.
- Use @JoinColumn(name = "profile_id") on the owning side.
- Use mappedBy on the inverse side to avoid creating two foreign keys.

B. One-to-Many & Many-to-One (@OneToMany , @ManyToOne)

- **Example:** Customer ↔ Orders.
- **Best Practice:** Always map both sides (bidirectional). The "Many" side (Order) should own the relationship (hold the foreign key).

C. Many-to-Many (@ManyToMany)

- **Example:** Student ↔ Course.
- Requires a separate join table.
- Use @JoinTable to define the intermediate table name and columns.

A. One-to-One (@OneToOne)

A **One-to-One** relationship means that one record in a database table is related to exactly **one** record in another table.

Common examples:

- **User ↔ UserProfile:** A user has one detailed profile.

- **Employee ↔ Parking Spot:** An employee is assigned exactly one spot.
- **Person ↔ Passport:** A person holds one active passport.

Here is the detailed breakdown of how to configure this in Spring Data JPA.

1. The Database Concept (How it maps)

Before writing Java code, you must decide which table will hold the **Foreign Key (FK)**.

- **The Owning Side:** The table that contains the foreign key column.
- **The Inverse (Referencing) Side:** The table that does *not* have the foreign key but wants to access the data.

Example Schema:

We have a `users` table and a `profiles` table.

- We put `profile_id` inside the `users` table.
 - Therefore, `User` is the **Owning Side**.
-

2. Bidirectional Mapping (The Standard Approach)

In a bidirectional relationship, you can fetch the Profile from the User *and* the User from the Profile.

A. The Owning Side (User)

This class holds the foreign key. We use `@JoinColumn` here.

```
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "users")
```

```

@Getter @Setter @NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;

    // 1. Marks the relationship
    // 2. CascadeType.ALL: If you save/delete User, save/delete Profile too.
    // 3. JoinColumn: Creates column 'profile_id' in the 'users' table.
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id", referencedColumnName = "id")
    private UserProfile profile;
}

```

B. The Inverse Side (UserProfile)

This class does *not* hold the foreign key. It just maps back to the User. We use `mappedBy` here.

```

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "user_profiles")
@Getter @Setter @NoArgsConstructor
public class UserProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

```

```

private Long id;

private String address;
private String phoneNumber;

// 'mappedBy' tells Hibernate: "I don't own this relationship."
// "Look at the 'profile' field in the User class to find the configuration."
@OneToOne(mappedBy = "profile")
private User user;
}

```

3. Deep Dive: Key Annotations Explained

@OneToOne

Defines the relationship type.

- **cascade = CascadeType.ALL** : Essential for One-to-One. It means if you create a `User` and set a `Profile` on it, you only need to save the `User`. Hibernate will automatically save the `Profile` for you.
- **fetch = FetchType.LAZY** : (Recommended) The associated entity is loaded only when you ask for it (e.g., `user.getProfile()`). The default is **EAGER**, which loads it immediately.

@JoinColumn(name = "profile_id")

- This annotation **must** go on the **Owning Side**.
- It tells Hibernate: *"Create a column named `profile_id` in this table (users) that points to the other table."*

mappedBy = "profile"

- This annotation goes on the **Inverse Side**.
- It tells Hibernate: *"Do not create a foreign key in my table. This relationship is already managed by the `profile` field in the other class."*

- **Crucial:** If you forget `mappedBy` on the inverse side, Hibernate will try to create foreign keys in **both** tables (or a join table), resulting in a messy schema.

B. One-to-Many & Many-to-One (`@OneToMany` , `@ManyToOne`)

1. The Mapping Logic

In JPA, this relationship is almost always **bidirectional**, meaning you can navigate from the "One" side to the "Many" side and vice versa.

- **The Many Side (The Child):** This is **always** the "Owning Side." The database table for this entity holds the Foreign Key (FK) column.
- **The One Side (The Parent):** This is the "Inverse Side." It uses the `mappedBy` attribute to point back to the child.

2. Complete Code Example

Let's use the **Library** (One) and **Book** (Many) analogy.

A. The Owning Side: Book (`@ManyToOne`)

The `Book` table will have a `library_id` column.

```
import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter @Setter @NoArgsConstructor
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```



```

private String title;

// ManyToOne is the OWNING side.
// It creates the foreign key 'library_id' in the 'book' table.
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "library_id", nullable = false)
private Library library;
}

```

B. The Inverse Side: Library (@OneToMany)

The `Library` class has a collection of books but doesn't manage the FK.

```

import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Getter @Setter @NoArgsConstructor
public class Library {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // 'mappedBy' refers to the 'library' field name in the Book class.
    // fetch = FetchType.LAZY is default (and best) here.
    @OneToMany(mappedBy = "library", cascade = CascadeType.ALL, orphan
Removal = true)
    private List<Book> books = new ArrayList<>();
}

```

```
// HELPER METHOD: Critical for keeping both sides of the relationship in sync in memory
public void addBook(Book book) {
    books.add(book);
    book.setLibrary(this);
}
}
```

3. Key Attributes Explained

`mappedBy = "library"`

This is the most important part of the `@OneToMany` annotation. It tells Hibernate: *"I am not the owner of this relationship. The field named 'library' in the Book class is handling the database column."*

`orphanRemoval = true`

If you remove a `Book` from the `Library.books` list (e.g., `library.getBooks().remove(0)`), Hibernate will automatically delete that book from the database. Without this, the book remains in the DB with a `NULL` `library_id`.

`CascadeType.ALL`

If you save a `Library` object that contains 5 `Book` objects in its list, calling `libraryRepository.save(library)` will automatically save all 5 books to the database in one go.

C. Many-to-Many (`@ManyToMany`)

A **Many-to-Many** relationship is used when multiple records in one table are associated with multiple records in another.

Real-world examples:

- **Students ↔ Courses:** A student can enroll in many courses, and a course can have many students.

- **Posts ↔ Tags:** A blog post can have many tags (Java, Spring, Web), and a tag can be applied to many posts.
-

1. The Database Logic: The Join Table

In a relational database, you cannot represent a Many-to-Many relationship with a simple foreign key in one of the tables. You **must** use a third table, called a **Join Table** (or Link Table).

This table typically contains two columns:

1. `student_id` (pointing to the Students table)
 2. `course_id` (pointing to the Courses table)
-

2. Implementation Code Example

Let's use the **Student** and **Course** example.

A. The Owning Side: Student

The "Owning Side" is where we define the join table details.

```
import jakarta.persistence.*;
import lombok.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "students")
@Getter @Setter @NoArgsConstructor
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

private String name;

@ManyToMany(fetch = FetchType.LAZY, cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(
    name = "student_courses",          // Name of the join table
    joinColumns = @JoinColumn(name = "student_id"), // FK for this entity
    inverseJoinColumns = @JoinColumn(name = "course_id") // FK for the other entity
)
private Set<Course> courses = new HashSet<>();

// Helper method
public void addCourse(Course course) {
    this.courses.add(course);
    course.getStudents().add(this);
}
}

```

B. The Inverse Side: Course

The inverse side uses `mappedBy` to point back to the field in the `Student` class.

```

@Entity
@Table(name = "courses")
@Getter @Setter @NoArgsConstructor
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @ManyToMany(mappedBy = "courses") // Matches the field name in Student class
}

```

```
private Set<Student> students = new HashSet<>();  
}
```

3. Critical Details to Remember

1. Set vs. List

Always prefer `Set<Entity>` over `List<Entity>` for `@ManyToMany`.

- **Why?** Hibernate is much more efficient with Sets. If you use a `List` and remove one item, Hibernate might delete *all* rows in the join table for that parent and re-insert the remaining ones. With a `Set`, it just deletes the single specific row.

2. The `mappedBy` Attribute

Just like in One-to-Many, `mappedBy` tells JPA that the mapping is already defined in the other class. Without it, JPA would create **two** separate join tables, which is almost never what you want.

3. Cascading

Be careful with `CascadeType.REMOVE` in Many-to-Many.

- If you delete a **Student**, you usually want to remove their entries from the join table, but you **do not** want to delete the **Course** itself (because other students are still enrolled in it).
- Use `CascadeType.PERSIST` and `CascadeType.MERGE` instead of `ALL`.

4. When the Join Table needs extra columns

Sometimes the join table needs more than just two IDs. For example, a `Registration` table might need a `registration_date` or a `grade` column.

In this case, **you cannot use** `@ManyToMany`. Instead, you must:

1. Create a new Entity for the join table (e.g., `Enrollment`).
2. Map it as `@ManyToOne` to Student.
3. Map it as `@ManyToOne` to Course.

Mapping Table

1. When to use the "Link Entity" (Two @ManyToOne)

You should stop using @ManyToOne and switch to the Link Entity approach if:

1. **Extra Columns:** You need to store data *about* the relationship (e.g., `date_joined`, `status`, `grade`, `price_at_purchase`).
2. **Audit Logs:** You want to know who created the link or when it was last updated.
3. **Business Logic:** The relationship itself is a "thing" in your business (e.g., an `Enrollment`, a `Subscription`, or an `OrderItem`).

The Mapping Logic

Instead of `Student ↔ Course`, you create:

`Student (1) ↔ (N) Enrollment (N) ↔ (1) Course`

2. Complete Example: The Enrollment Entity

In this scenario, we want to know **when** a student enrolled in a course.

Step 1: The Student (One side)

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "student")
    private List<Enrollment> enrollments;
}
```

Step 2: The Course (One side)

```
@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @OneToMany(mappedBy = "course")
    private List<Enrollment> enrollments;
}
```

Step 3: The Enrollment (The Link Entity - Many side)

This table now holds the foreign keys for both Student and Course, plus our extra data.

```
@Entity
@Table(name = "enrollments")
public class Enrollment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    // EXTRA DATA
    private LocalDateTime enrollmentDate;
```

```
private String grade;
}
```

Fetching:

1. The Two Types of Fetching

There are only two options for the `fetch` attribute in JPA annotations (like `@OneToOne`, `@OneToMany`, etc.).

A. `FetchType.EAGER` (Load Immediately)

- **Behavior:** When you load the parent entity, Hibernate **immediately** fires a query (or a JOIN) to fetch the related child entity.
- **Pros:** Data is available instantly. No risk of `LazyInitializationException`.
- **Cons:** Can be very slow. If you load 100 Users, and each has a Profile, Eager loading might fetch all Profiles even if you don't need them.

B. `FetchType.LAZY` (Load on Demand)

- **Behavior:** When you load the parent entity, Hibernate fetches **only the parent**. The child field is replaced by a "Proxy" (a placeholder). The real data is fetched from the DB **only when you call the getter** (e.g., `user.getProfile()`).
- **Pros:** Much better performance. Saves memory.
- **Cons:** If you try to access the child after the database session is closed (e.g., inside the View/Controller layer), you get a `LazyInitializationException`.

2. Default Defaults (Important!)

If you do not specify a fetch type, JPA uses these defaults:

Relationship	Default Fetch Type	Reasoning
<code>@OneToOne</code>	EAGER	Loading one single related row is usually cheap.
<code>@ManyToOne</code>	EAGER	Loading one single related row is usually cheap.

@OneToMany	LAZY	Loading a list of thousands of items is expensive!
@ManyToMany	LAZY	Loading a list of thousands of items is expensive!

3. Code Example

Let's look at a scenario with `User` and `Post`. A User can have many Posts.

The Entity Setup

```
@Entity
public class User {
    @Id
    private Long id;
    private String name;

    // We explicitly set this to LAZY (Best Practice)
    // Default is LAZY anyway, but good to be explicit.
    @OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
    private List<Post> posts;
}
```

Scenario A: How `FetchType.LAZY` works

(This is how `@OneToMany` behaves by default)

```
// 1. We call the repository
User user = userRepository.findById(1L).get();
// SQL Fired: SELECT * FROM user WHERE id = 1;
// At this point, user.posts is NOT loaded. It is a Proxy.

System.out.println(user.getName()); // Works fine.

// 2. We access the relationship
List<Post> myPosts = user.getPosts();
```

```
// Still no SQL fired yet...

// 3. We touch the data inside the list
int count = myPosts.size();
// SQL Fired NOW: SELECT * FROM posts WHERE user_id = 1;
```

Key Takeaway: If you never call `myPosts.size()` or access the list, the second query is **never executed**. This saves resources.

Scenario B: How `FetchType.EAGER` works

If we changed the annotation to:

```
@OneToMany(mappedBy = "user", fetch = FetchType.EAGER)
```

```
// 1. We call the repository
User user = userRepository.findById(1L).get();

// Hibernate immediately fires a JOIN or a second SELECT
// SQL: SELECT * FROM user WHERE id = 1;
// SQL: SELECT * FROM posts WHERE user_id = 1;

// The data is already in memory.
System.out.println(user.getPosts().size()); // No new DB query here.
```

4. Visualizing the Difference (Console Logs)

If you enable SQL logging (`spring.jpa.show-sql=true`), here is exactly what you see:

EAGER Output:

```
Hibernate: select u1_0.id, u1_0.name from users u1_0 where u1_0.id=?
Hibernate: select p1_0.user_id, p1_0.id, p1_0.title from posts p1_0 where p1_0.user_id=?
```

(Both happen immediately)

LAZY Output:

```
Hibernate: select u1_0.id, u1_0.name from users u1_0 where u1_0.id=?  
-- Application runs some logic... --  
-- Application calls user.getPosts().size() --  
Hibernate: select p1_0.user_id, p1_0.id, p1_0.title from posts p1_0 where p1_0.user_id=?
```

(The second query waits until it is needed)

5. When to use which?

1. **Always prefer** **LAZY** for collections (**@OneToMany** , **@ManyToMany**).
 - *Why?* You rarely need every single order history when you just want to check a user's name.
2. **Use** **LAZY** for **@OneToOne** and **@ManyToOne** if the related object is heavy (e.g., contains a Blob/Image).
3. **Use** **EAGER** only if you are 100% sure you will *always* need that data every time you load the parent, and the dataset is small.

Cascading:

In JPA, **Cascading** allows you to propagate (pass on) state changes from a **Parent** entity to its **Children**.

Without cascading, if you save a **Parent** , JPA will ignore the **Child** objects inside it, often leading to errors like **IllegalStateException** or data simply not being saved.

1. How to Define Cascading

Cascading is defined inside the relationship annotations (**@OneToOne** , **@OneToMany** , etc.) using the **cascade** attribute.

```
@OneToMany(mappedBy = "user", cascade = CascadeType.PERSIST)
private List<Post> posts;
```

2. All Cascade Types Explained

A. `CascadeType.PERSIST` (Create)

When you save (persist) the Parent, all associated Children are also saved.

- **Scenario:** You create a new `User` and a new `Profile` object. You only call `userRepository.save(user)`.
- **Result:** Both the user and the profile are inserted into the database.

B. `CascadeType.MERGE` (Update)

If the Parent is updated (merged), the Children are updated as well.

- **Scenario:** You fetch an existing `User` from the DB, change their name, and also change the text of one of their `Posts`. You call `userRepository.save(user)`.
- **Result:** Hibernate issues an `UPDATE` statement for both the user and the modified post.

C. `CascadeType.REMOVE` (Delete)

When the Parent is deleted, all its Children are deleted automatically.

- **Scenario:** You delete a `User`.
- **Result:** All `Posts` belonging to that user are deleted. **Be careful with this!** In a Many-to-Many relationship, this could delete data you didn't intend to lose.

D. `CascadeType.REFRESH` (Sync from DB)

If you refresh the Parent (re-read it from the database), all Children are also re-read to ensure they match the database state.

- **Scenario:** Someone else changed the data in the DB while your application was running. You call `entityManager.refresh(user)`.
- **Result:** The `User` and all their `Posts` are reloaded from the latest DB snapshot.

E. `CascadeType.DETACH` (Remove from Session)

Detaching an entity means it is no longer managed by the Hibernate session (changes won't be tracked). This cascades that "unmanaged" status to children.

F. `CascadeType.ALL` (The Bundle)

This is a shorthand for applying **all of the above** types. It is the most commonly used type for **One-to-One** and **One-to-Many** (Parent-Child) relationships.

3. The "Secret" Type: `orphanRemoval = true`

While not technically a `CascadeType`, it acts like one. It is used exclusively in `@OneToMany` and `@OneToOne`.

- **Difference from `CascadeType.REMOVE`:**
 - `REMOVE` only triggers when you delete the **Parent**.
 - `orphanRemoval` triggers when you **remove a child from the parent's list**.

Example:

```
// Inside User Entity
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Post> posts;

// In your Service
user.getPosts().remove(0);
userRepository.save(user);
// Because of orphanRemoval=true, the post is DELETED from the database.
// Without it, the post would just sit in the DB with a NULL user_id.
```

4. Best Practices Summary

Relationship	Recommended Cascade	Why?
--------------	---------------------	------

One-to-One	<code>ALL</code>	Usually, the profile/identity cannot exist without the user.
One-to-Many	<code>ALL</code> or <code>PERSIST, MERGE</code>	Makes saving complex objects easier.
Many-to-One	NONE (Usually)	If you delete a <code>Book</code> , you definitely don't want to delete the <code>Library</code> it belongs to!
Many-to-Many	<code>PERSIST, MERGE</code>	Never use <code>REMOVE</code> or <code>ALL</code> here, or deleting one Student might delete all Courses, which then deletes all other Students enrolled in them!